High-Performance Star-M SVD for Big Data Compression

Md Taufique Hussain
Wake Forest University
Winston-Salem, NC, USA
hussaint@wfu.edu

Grey Ballard
Wake Forest University
Winston-Salem, NC, USA
ballard@wfu.edu

Aditya Devarakonda
Wake Forest University
Winston-Salem, NC, USA
devaraa@wfu.edu

Srinivas Eswar
Argonne National Lab
Lemont, IL, USA
seswar@anl.gov

Naman Pesricha
Indian Institute of Science
Bengaluru, India
namanp@iisc.ac.in

Vishwas Rao
Argonne National Lab
Lemont, IL, USA
vhebbur@anl.gov

Abstract

In the era of big data, effectively compressing large datasets while performing complex mathematical operations is crucial. Tensor-based decomposition methods have shown superior compression capabilities with minimal loss of accuracy compared to traditional matrix methods. Under the $\star_{\mathbf{M}}$ tensor framework, tensors can be decomposed in a matrix-mimetic way, including using the $\star_{\mathbf{M}}$ SVD. This tensor SVD has optimality guarantees and has shown exceptional performance on specific types of data, but software implementations have been mostly limited to productivity-oriented languages. In this work, we present our development of a shared-memory parallel, high-performance solution designed to efficiently implement the underlying algorithms. This software will enable optimal compression of extensive scientific datasets, paving the way for enhanced data analysis and insights.

CCS Concepts

• **Computer systems organization → Parallel architectures**.

Keywords

parallel computing, high-performance computing, tensor factorizations

ACM Reference Format:
Md Taufique Hussain, Grey Ballard, Aditya Devarakonda, Srinivas Eswar, Naman Pesricha, and Vishwas Rao. 2026. High-Performance Star-M SVD for Big Data Compression. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP '26), September 28–October 01, 2026, Singapore, Singapore.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3832810.3832889

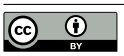
1 Introduction

Truncated tensor-based singular value decompositions based on the star-M ($\star_{\mathbf{M}}$) tensor-tensor product mimic the matrix SVD [26].

An input tensor can be decomposed into the product of three factor tensors with properties of orthogonality and structure that are analogous to the matrix case. Furthermore, this decomposition can be truncated for the purpose of data compression, for example, in a systematic way to determine the optimal representation given either an error tolerance or specified rank parameters. This is possible due to the existence of theoretical guarantees established for the $\star_{\mathbf{M}}$ tensor algebra that generalize the Eckart-Young theorem [12] for low-rank matrix approximation.

As described in more detail in Section 2, given 3-way input tensors, the $\star_{\mathbf{M}}$ product is parametrized by an invertible matrix $\mathbf{M}$ and defined by a transformation of the mode-3 fibers of the inputs by $\mathbf{M}$, independent matrix multiplications of the mode-3 slices, and a back transformation of the output mode-3 fibers by $\mathbf{M}^{-1}$. The $\star_{\mathbf{M}}$ truncated SVD (t-SVD) is thus based on slice-wise matrix SVDs after transformation by the matrix $\mathbf{M}$, and it is particularly advantageous when the transformation yields many slices to be either low-rank or insignificant relative to other slices. The $\star_{\mathbf{M}}$ t-SVD is not agnostic with respect to mode ordering, and the choice of $\mathbf{M}$ can have a substantial effect on the compressibility of a data set. Implementations of $\star_{\mathbf{M}}$ algebra and the $\star_{\mathbf{M}}$ t-SVD exist in MATLAB and Python—see [24, 29, 31], for example—but no high performance implementations have been developed. High performance, parallel software exists for many other tensor formats including CANDECOMP/PARAFAC (CP) [13, 17, 37], Tucker [4, 22], and Tensor Train [2, 34]. The goal of this work is to design an efficient and scalable shared-memory parallel implementation of the fundamental algorithms for computing $\star_{\mathbf{M}}$ t-SVD for general multiway data, enabling its use for datasets that were otherwise too large to handle using existing techniques.

The core of the $\star_{\mathbf{M}}$ t-SVD algorithm is a "batch" of independent matrix SVDs, all of the same dimension, which is ripe for parallelization. In fact, implementations of a batched SVD are emerging in standard libraries like MKL [19, Version 2024.1] and MAGMA [11, Version 2.10.0]. However, the (back-)transformations of the slices necessarily happens in the orthogonal direction, across slices, enabling the decoupling. We show in Section 3 that the transformations can be parallelized via efficient tensor-times-matrix operations that respect the data layout of the tensor in memory and exploits the interface to the batched BLAS [1]. Two primary algorithms exist for t-SVD: the first specifies a constant rank for the truncation of all slices, and the second specifies an error tolerance that guides the truncation of various ranks across slices. We show how to parallelize the first algorithm using the expert interface of LAPACK, and we explore computation-memory tradeoffs in the design of the second algorithm, which requires analysis of the computed slice-wise singular values to determine ranks.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICPP '26, Singapore, Singapore
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2657-6/26/09
https://doi.org/10.1145/3832810.3832889

In Section 4, we consider data sets coming from the domains of climate reanalyses, computational fluid dynamics, and X-ray crystallography. We describe how mode ordering and choice of transformation matrix can affect both performance of the parallel implementations and the quality of the compressed representation. In the case of the climate data, we show the benefits of using tensor-based compression over the standard matrix-based approach used in that field. Our parallel design decisions enable efficient parallel scaling, and we show that on a platform with 64 cores, these datasets can be compressed up to 42×, 40×, and 30× faster than a single core, respectively. Our implementation (publicly available at https://github.com/Streaming-Tensors/pystarm/) enables users to observe speedups of up to 1000× over existing MATLAB code on a similar machine. Thus, this implementation allows domain scientists to analyze larger data sets in less time, enabling the $\star_{\mathbf{M}}$ t-SVD to be applied as broadly as other tensor decomposition techniques.

2 Background

This section reviews the mathematical and algorithmic background relevant to our work. We first define notation for tensor and matrix-tensor operations, and define the $\star_{\mathbf{M}}$-product (Section 2.1). We then introduce the $\star_{\mathbf{M}}$ t-SVD together with its full (t-SVDM-I) and adaptively truncated (t-SVDM-II) variants, whose parallelization and evaluation is the main focus of our work (Section 2.2). Finally, we summarize the Eckart-Young-style optimality results that distinguish the $\star_{\mathbf{M}}$ t-SVD from competing tensor decompositions (Section 2.3) and motivate the need for a shared-memory parallel implementation suited to large scientific datasets (Section 2.4).

2.1 Notation

We use uppercase bold letters (e.g., $\mathbf{M}$) to denote matrices and calligraphic letters (e.g., $\mathcal{A}$) to denote tensors. A three-way tensor $\mathcal{A} \in \mathbb{R}^{m\times p\times n}$ has n frontal slices $\mathcal{A}_{:,:,i} \in \mathbb{R}^{m\times p}$ for $i = 1, \ldots, n$. The mode-k unfolding $\mathbf{A}_{(k)}$ arranges the mode-k fibers of $\mathcal{A}$ as columns of a matrix. The mode-k tensor times matrix product (TTM), written $\mathcal{B} = \mathcal{A} \times_k \mathbf{M}$ for $\mathbf{M} \in \mathbb{R}^{J\times n_k}$, is defined such that

$$\mathbf{B}_{(k)} \;=\; \mathbf{M}\,\mathbf{A}_{(k)}.$$

The result $\mathbf{B}$ has the same shape as $\mathcal{A}$ except that mode k has size J instead of n_k.

$\star_{\mathbf{M}}$-Product. The $\star_{\mathbf{M}}$-product, introduced by Kilmer et al. [26], is a tensor-tensor multiplication parameterized by an invertible matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$. Given $\mathcal{A} \in \mathbb{R}^{m\times p\times n}$ and $\mathcal{B} \in \mathbb{R}^{p\times \ell\times n}$, the $\star_{\mathbf{M}}$-product $\mathcal{C} = \mathcal{A} \star_{\mathbf{M}} \mathcal{B} \in \mathbb{R}^{m\times \ell\times n}$ is defined in three steps. First, both operands are transformed to the $\mathbf{M}$-domain by applying $\mathbf{M}$ along mode 3,

$$\hat{\mathcal{A}} = \mathcal{A} \times_3 \mathbf{M}, \qquad \hat{\mathcal{B}} = \mathcal{B} \times_3 \mathbf{M}.$$

Second, the n pairs of frontal slices are multiplied independently in the transform domain,

$$\hat{\mathcal{C}}_{:,:,i} \;=\; \hat{\mathcal{A}}_{:,:,i}\,\hat{\mathcal{B}}_{:,:,i}, \qquad i = 1, \ldots, n,$$

where each product is an ordinary matrix multiplication. Third, the result is mapped back to the original domain by applying $\mathbf{M}^{-1}$ along mode 3,

$$\mathcal{C} \;=\; \hat{\mathcal{C}} \times_3 \mathbf{M}^{-1}.$$

When $\mathbf{M}$ is a nonzero scalar multiple of an orthogonal matrix, $\mathbf{M}^{-1}$ reduces to a (scaled) transpose, and the $\star_{\mathbf{M}}$-product inherits matrix-mimetic properties such as an associative and distributive algebra, a notion of orthogonality and identity tensors, and the Eckart-Young-style optimality results which are proved by Kilmer et al. [26].

2.2 Tensor Singular Value Decomposition

In this section, we provide a brief overview of the mathematical background and algorithms that underpin our work. We focus on the tensor singular value decomposition (t-SVD) for compressing scientific datasets. The t-SVD can be defined in many ways, but we focus on the $\star_{\mathbf{M}}$ t-SVD framework developed by Kilmer et al. [26], which is defined with respect to an invertible matrix $\mathbf{M}$. Given an invertible matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$ and a 3-way tensor $\mathcal{A} \in \mathbb{R}^{m\times p\times n}$, the $\star_{\mathbf{M}}$ t-SVD of $\mathcal{A}$ is the factorization $\mathcal{A} = \mathcal{U} \star_{\mathbf{M}} \mathcal{S} \star_{\mathbf{M}} \mathcal{V}$, where $\mathcal{U}$ and $\mathcal{V}$ are orthogonal tensors and $\mathcal{S}$ is an f-diagonal tensor (i.e., diagonal frontal slices). A key computational step is transforming $\mathcal{A}$ to the $\mathbf{M}$-domain by applying $\mathbf{M}$ to mode 3,

$$\hat{\mathcal{A}} = \mathcal{A} \times_3 \mathbf{M},$$

after which the frontal slices $\hat{\mathcal{A}}_{:,:,i}$ are mutually decoupled and each can be independently factored by a standard thin matrix SVD. The output factor tensors $\hat{\mathcal{U}}, \hat{\mathcal{S}}, \hat{\mathcal{V}}$ are then mapped back to the original domain by applying $\mathbf{M}^{-1}$ to mode 3. The full $\star_{\mathbf{M}}$ t-SVD is given in Algorithm 1, and the adaptive truncated variant with relative error tolerance ε is given in Algorithm 2 (see Figure 1). Extensions to 4th and higher order tensors are straightforward. Transformation to/from the $\mathbf{M}$-domain occurs via TTMs with $\mathbf{M}_i/\mathbf{M}_i^{-1}$ for $i \geq 3$.

Choice of $\boldsymbol{M}$. The transform matrix $\mathbf{M}$ is an important decision point in the $\star_{\mathbf{M}}$ framework. Since $\mathbf{M}$ determines the basis used to decouple the frontal slices, it directly affects the singular-value distribution and the resulting compression quality. Prior work has considered fixed analytic transforms (e.g. DCT and DFT) [26], data-dependent transforms computed from $\mathcal{A}$ (e.g. singular vectors of the mode-3 unfolding), and learned transforms obtained by optimization [31].

Kilmer et al. [26] survey several fixed, analytic choices that inherit the Eckart-Young optimality guarantee of the $\star_{\mathbf{M}}$ t-SVD framework. The (normalized) discrete Fourier transform (DFT) recovers the original t-product of Kilmer and Martin and allows an $O(n \log n)$ application to each fiber via the FFT, but yields complex-valued frontal slices in the transform domain. The discrete cosine transform (DCT) is a real-valued alternative that can also be applied with an $O(n \log n)$ algorithm.

The matrix $\mathbf{M}$ can also be tailored to the input tensor. A simple data-driven choice discussed in [26] is to take $\mathbf{M}$ as the (transposed) matrix of left singular vectors of the mode-3 unfolding $\mathbf{A}_{(3)}$, which empirically yields better approximation at a given truncation rank than other fixed transforms. However, this data-driven approach comes at the cost of an upfront SVD on the unfolding.

More recently, Newman and Keegan [31] have proposed a variable projection (VarPro) approach that jointly optimizes $\mathbf{M}$ and the truncated factors. The inner slicewise SVD problem is solved in closed form as a function of $\mathbf{M}$ and projected out, leaving a reduced nonlinear optimization problem. This produces transforms tailored

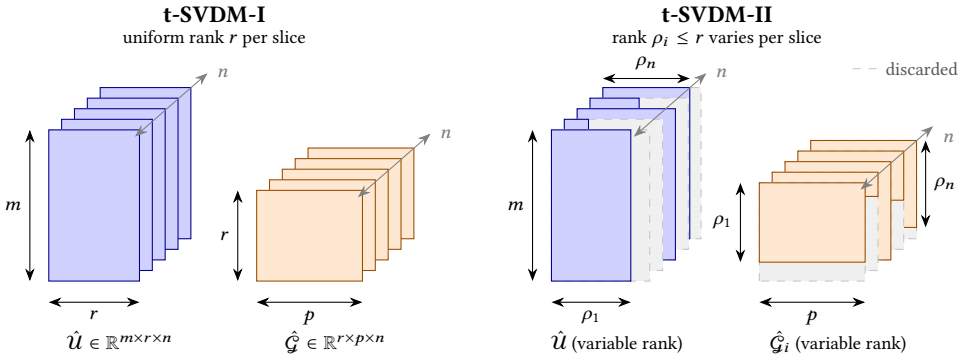


Figure 1: The output factor structure of t-SVDM-I (left) has uniform ranks across frontal slices which yield coarser error control, whereas t-SVDM-II (right) allows for variable frontal slice ranks ρ_i which can adapt to approximation requirements.

to specific datasets and downstream tasks, at the cost of an iterative training step. Our work treats $\mathbf{M}$ as an arbitrary user-supplied invertible matrix, so any of the above choices can be utilized while leveraging the speedups achievable through shared-memory parallelization.

Algorithm 1 Full t-SVDM of $\mathcal{A}$

Require: $\mathcal{A} \in \mathbb{R}^{m\times p\times n}$, invertible $\mathbf{M} \in \mathbb{R}^{n\times n}$
Ensure: $\mathcal{U}, \mathcal{S}, \mathcal{V}$ such that $\mathcal{A} = \mathcal{U} \star_{\mathbf{M}} \mathcal{S} \star_{\mathbf{M}} \mathcal{V}^\top$
1: $\hat{\mathcal{A}} \leftarrow \mathcal{A} \times_3 \mathbf{M}$
2: **for** $i = 1, \ldots, n$ **do**
3: $\quad[\hat{\mathcal{U}}_{:,:,i},\ \hat{\mathcal{S}}_{:,:,i},\ \hat{\mathcal{V}}_{:,:,i}] \leftarrow \mathrm{svd}(\hat{\mathcal{A}}_{:,:,i})$
4: **end for**

Algorithm 2 t-SVDM-II with relative error tolerance ε

Require: Tensor $\mathcal{A} \in \mathbb{R}^{m\times p\times n}$, $\mathbf{M} \in \mathbb{R}^{n\times n}$ a nonzero multiple of an orthogonal matrix, relative error tolerance $\varepsilon \in (0, 1)$
Ensure: Per-face truncation levels $\rho = (\rho_1, \ldots, \rho_n)$ and truncated factors $\{\hat{\mathcal{U}}_{:,1:\rho_i,i}, \hat{\mathbf{G}}_{\rho_i}\}_{i=1}^n$
1: Compute the t-SVDM of $\mathcal{A}$
2: $v \leftarrow \mathrm{concatenate}\Big((\hat{\mathcal{S}}_{j,j,i})^2 \text{ for all } i, j\Big)$
3: $v \leftarrow \mathrm{sort}(v,\ \text{ascending})$
4: Let w be the cumulative-sum vector, i.e., $w_k = \sum_{\ell=1}^{k} v_\ell$
5: Find the largest index J such that $w_J / \|\hat{\mathcal{S}}\|_F^2 < \varepsilon^2$
6: $\tau \leftarrow v_J$ ▷ largest singular value to be discarded
7: **for** $i = 1, \ldots, n$ **do**
8: $\quad\rho_i \leftarrow$ number of singular values of $\hat{\mathcal{A}}_{:,:,i}$ greater than τ
9: $\quad$Keep only $\hat{\mathcal{U}}_{:,1:\rho_i,i}$ and form $\hat{\mathbf{G}}_{\rho_i} \leftarrow \hat{\mathcal{S}}_{1:\rho_i,1:\rho_i,i}\, \hat{\mathcal{V}}^\top_{:,1:\rho_i,i}$
10: **end for**

2.3 Optimality of $\star_{\mathbf{M}}$ t-SVD

The primary advantage of the $\star_{\mathbf{M}}$ t-SVD is that it admits an Eckart-Young-like optimality guarantee whereas established tensor decompositions such as the CANDECOMP/PARAFAC (CP) [8, 16, 18], Tucker [36], Tensor-Train SVD [32], and the higher-order SVD (HOSVD) [9] families do not (see also the survey by Kolda and Bader [27]). For matrices, the Eckart-Young theorem states that truncating the SVD to its leading rank-k singular values and vectors yields the best rank-k approximation in the Frobenius (and 2-) norm [12, 30]. This single property is what makes the matrix SVD the primary tool for dimensionality reduction and lossy compression. Generalizing this guarantee to multiway data has been a long-standing open problem.

For the CP decomposition, determining the tensor rank is NP-hard in general, and the factor matrices are not required to be orthogonal or even full rank [26]. As a result, no provably optimal truncation policy is known. Practical implementations often rely on iterative alternating-least-squares schemes that may converge slowly, are sensitive to initialization, and offer no global-optimality guarantees on the resulting approximation. The Tucker and HOSVD decompositions do produce factor matrices with orthonormal columns, but the core tensor is typically dense and does not yield an optimal low-rank approximation. Instead, the truncated HOSVD is only *quasi-optimal*: for a d-way tensor $\mathcal{A}$, an HOSVD-truncated approximation $\mathcal{B}_{\mathbf{k}}$ satisfies

$$\|\mathcal{A} - \mathcal{B}_{\mathbf{k}}\|_F \ \le\ \sqrt{d}\, \|\mathcal{A} - \mathcal{B}^\star\|_F,$$

where $\mathcal{B}^\star$ is the optimal multilinear-rank-$\mathbf{k}$ approximation [10]. The $\sqrt{d}$ factor compounds with tensor order, so the gap grows as the data become more high-dimensional.

In contrast, when $\mathbf{M} \in \mathbb{R}^{n\times n}$ is a nonzero scalar multiple of an orthogonal matrix, Kilmer et al. [26] prove that both the t-SVDM-I (uniform-rank) and t-SVDM-II (error-tolerance, variable-rank) truncations are *exactly* Eckart-Young optimal in the Frobenius norm under the framework induced by $\star_{\mathbf{M}}$ This work showed that truncating the singular values of the transformed slices directly produces the best lower-rank representation. Combined with the fact that the slice-wise SVDs in the $\mathbf{M}$-domain are decoupled (and easily parallelizable), the $\star_{\mathbf{M}}$ framework provides an attractive option for compressing large scientific datasets where low-rank optimality and computational efficiency are important.

2.4 Available Implementations of $\star_{\mathbf{M}}$ t-SVD

Despite the Eckart-Young-like optimality results, the practical impact of the $\star_{\mathbf{M}}$ t-SVD is limited by the available software. The

reference implementations accompanying [26] and the broader $\star_{\mathbf{M}}$ work are written in productivity-oriented languages, such as MATLAB and pure Python. While these implementations are excellent for small scale experimentation with the $\star_{\mathbf{M}}$ framework, they are not directly suitable for large-scale data compression tasks on modern multicore architectures. To our knowledge, no shared-memory parallel, high-performance implementation of the $\star_{\mathbf{M}}$ t-SVD currently exists. This is in contrast to the software maturity of the matrix SVD (e.g. LAPACK and ScaLAPACK), CP, and Tucker decompositions, for which both shared-memory and distributed-memory parallel libraries are available [3, 5, 33, 35].

This work closes the parallel algorithms design and software gap by developing the `pystarm` software package, a shared-memory parallel, high-performance implementation of the $\star_{\mathbf{M}}$ t-SVD that exposes parallelism in the tensor-times-matrix and the slicewise SVD operations. We utilize threaded batched-BLAS and LAPACK kernels through Intel MKL, and provide a Python interface so that performance and productivity are simultaneously achievable for domain scientists.

3 Design and Implementation

We implement the two algorithms described in Section 2 in C++ using Intel's Math Kernel Library (MKL) for dense linear algebra, with Python bindings provided via pybind11. We restrict $\mathbf{M}$ to be orthonormal, so $\mathbf{M}^{-1} = \mathbf{M}'$, eliminating the need for explicit matrix inversion. Recent work has extended the t-SVD framework to non-orthonormal transformations [23, 25], but orthonormal choices provide optimality guarantees while reducing computational complexity for backward transformations. All tensor and matrix data are stored in column-major order. In that way layout the $m \times p$ frontal slices of a 3-way tensor $\mathcal{A} \in \mathbb{R}^{m \times p \times n}$ occupy contiguous, non-overlapping memory regions, so each slice can be addressed as a matrix without data movement or transposition. Both algorithms decompose into two main computational kernels, tensor-times-matrix multiplication and slice-wise SVD, which we describe in turn before addressing the specifics of each variant.

We report some performance numbers of our implementation in this section and the next. We ran all experiments on a single compute node of ALCF Aurora supercomputer. A single compute node of the Aurora is equipped with a dual-socket Intel Xeon CPU machine having 52 cores per socket (104 cores total). We run our experiments by changing the number of threads by power of two. So we use upto 64 threads in our experiments.

3.1 Tensor-Times-Matrix

The TTM $\hat{\mathcal{A}} = \mathcal{A} \times_k \mathbf{M}$ is the workhorse of both t-SVD algorithms. For a 3-way tensor it applies $\mathbf{M}$ to mode 3 to enter the transform domain; for a d-way tensor it applies $\mathbf{M}$ to every mode $k > 2$ (with potentially a different transform per mode). The first two modes are never contracted by the algorithm, so we need TTM kernels only for modes $3, \ldots, d$. Among these, mode d is the *last mode* of the tensor and modes $3, \ldots, d-1$ are *middle modes*; the structure of the computation differs in the two cases.

For a TTM on the last mode of a d-way tensor, the natural-order layout exposes d^{th} unfolding of the tensor as a single $\left(\prod_{i<d} n_i\right) \times n_d$ matrix in row major layout; right-multiplying it by $\mathbf{M}' \in \mathbb{R}^{n_d \times J}$ produces the correctly contracted result and is a single level-3 BLAS GEMM call [6]. For a 3-way tensor this is the only TTM that arises ($k = d = 3$), so MKL's multi-threaded GEMM handles all parallelism automatically.

For a TTM on a middle mode k with $2 < k < d$, the mode-k fibers are not regularly spaced in memory, so the operation must be expressed as a collection of GEMM calls. Let $M_k = \prod_{i<k} n_i$ and $P_k = \prod_{i>k} n_i$. The tensor can be viewed as P_k contiguous submatrices of shape $M_k \times n_k$, each to be right-multiplied by $\mathbf{M}' \in \mathbb{R}^{n_k \times J}$ to yield an $M_k \times J$ result [6]. Middle-mode TTMs arise only when the tensor has more than three modes, but then they dominate: in the 6-way `ncep-air-6` dataset (described in Section 4.1.1), three of the four required TTMs (modes 3, 4, and 5) are middle-mode contractions. The efficient parallelization of these middle-mode TTMs is therefore an important design choice for a high-performance shared-memory implementation of the $\star_{\mathbf{M}}$ t-SVD.

3.1.1 *Multithreaded TTM Design Choices.*

We identify three primary choices to implement multithreaded TTM for the middle modes.

Choice 1: Loop over multithreaded GEMM. We can call the individual GEMM operations one after another sequentially and use optimized multithreaded GEMM kernels for each of the individual GEMMs. We call this the `loop` variant of TTM.

Choice 2: Parallel for with sequential GEMM. We can lift the parallelism from MKL multithreading to OpenMP multithreading by leveraging a parallel for over the batch of GEMM and use sequential GEMM calls. These calls can be issued simultaneously without synchronization. We call this the `parfor` variant of TTM.

Choice 3: Batched GEMM. We can use MKL's batched strided GEMM interface, in which a single MKL call performs all P_k multiplications such that: the input stride between successive submatrices is $M_k n_k$, the output stride is $M_k J$, and the stride for $\mathbf{M}$ is zero so the same matrix is reused across all batches. This allows MKL's scheduling layer to optimize across the batch, reducing per-call overhead compared to an equivalent loop over individual GEMM calls. For the mode-3 TTM that appears in both algorithms on a 3-way tensor, $P_k = 1$ and $M_k = mp$, so the batched call degenerates to a single GEMM that right-multiplies the $(mp) \times n$ matrix view of the tensor by $\mathbf{M}' \in \mathbb{R}^{n \times J}$, yielding an $(mp) \times J$ result. We call this the `batched` variant of TTM.

3.1.2 *Performance of three TTM designs.*

We compare the performance of the three design choices of TTM on the `ncep-air-6` tensor discussed in Section 4.1.1. It is a 6-way tensor and exercises both the middle-mode (modes 3, 4, 5) and last-mode (mode 6) cases. We show the comparison in Figure 2.

We observe that the `batched` variant works best for both middle and last modes. The `parfor` variant suffers in the case of last mode: the `parfor` variant reduces to a single sequential GEMM call, hence it does not scale at all with increasing thread count. The `loop` variant suffers in the cases of first few modes. As explained above, TTM on a middle mode reduces to multiple GEMM calls. For the first few modes, the `loop` variant reduces to multiple calls of multithreaded GEMM each having a small workload. Hence, the scalability suffers at higher thread counts. For the remainder of

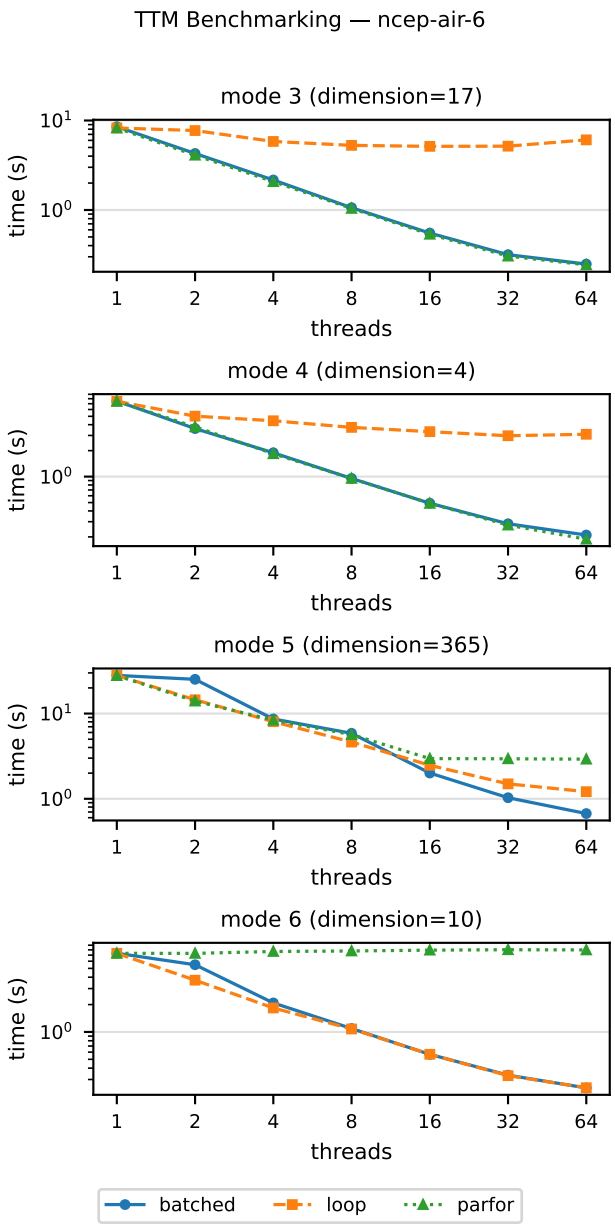


Figure 2: TTM performance of three variants (`batched` vs. `loop` vs. `parfor`) for the `ncep-air-6` dataset across thread counts and TTM modes.

this paper we report experiments with only the `batched` variant of TTM as it is the clear winner among all three design choices.

3.2 Slice-wise SVD

After transforming to the **M**-domain, both algorithms require computing the matrix SVD of each frontal slice $\hat{\mathcal{A}}_{:,:,i}$ independently. Because LAPACK's SVD routine overwrites its input, each slice is materialized as a fresh copy before factorization.

We support two factorization modes. The first computes the full thin SVD of each slice, yielding factor tensors $\hat{\mathcal{U}} \in \mathbb{R}^{m\times r\times n}$ and $\hat{\mathcal{V}} \in \mathbb{R}^{p\times r\times n}$ (where $r = \min(m, p)$) together with a matrix of singular values. The second computes only the leading k singular triplets via a truncated SVD, avoiding the cost of forming the full factorization when $k \ll r$.

3.2.1 Multithreaded Design Choices. The n slice SVDs are independent, presenting two natural parallelization strategies.

Choice 1: Sequential slices, parallel SVD. Each LAPACK call is itself multi-threaded, exploiting intra-slice parallelism through MKL's internal threading. We call this the `loop` variant.

Choice 2: Parallel slices, sequential SVD. In the second, a parallel loop distributes slices across threads, with each thread calling a single-threaded LAPACK routine. We call this the `parfor` variant.

The first strategy is preferable when the slices are large relative to the number of slices ($m, p \gg n$), so each SVD has enough work to keep many threads busy; the second is preferable when there are many small slices ($n \gg m, p$), where inter-slice parallelism offers more opportunity than intra-slice parallelism.

3.2.2 Performance of the designs. In Figure 3 we show a microbenchmark demonstrating the performance of the two design choices of slicewise SVD on the three practical tensors described in Section 4. We observe that the `parfor` variant gives almost linear scalability over increasing thread count. In contrast, the `loop` variant does not scale. We believe that this is due to the small sizes of the SVD where parallel overhead dominates. Given the complexity of parallelizing the SVD computation, we expect the `parfor` variant to yield better performance for most applications.

3.3 Full t-SVDM and t-SVDM-I

Full t-SVDM. Algorithm 1 shows the full t-SVDM algorithm. For simplicity, the algorithm is shown for a three-way tensor. This variant computes all singular values and vectors of each frontal slice.

In contrast, t-SVDM-I computes a truncated SVD with fixed rank-k for each frontal slice. This variant enables predictable performance in each SVD call and reduces the overhead of computing variable rank SVDs and the complexity of determining ranks adaptively. The full SVD uses the `gesvd` LAPACK call, whereas the truncated SVD calls `gesvdx`.

3.4 t-SVDM-II

The adaptive truncated variant (Algorithm 2) introduces per-slice truncation ranks ρ_i that vary across slices and are not known until after the singular values have been inspected globally. This data dependency rules out the simple composition used for t-SVDM-I and motivates three strategies with different trade-offs between computational redundancy and memory efficiency.

3.4.1 Multithreaded Design Choices. All strategies consist of three stages: (1) compute the singular values of all slices, (2) determine the threshold τ, and (3) compute the final factorization.

Choice 1: Truncate. The truncation strategy is a straightforward way to compute the t-SVDM-II factorization. We compute the full

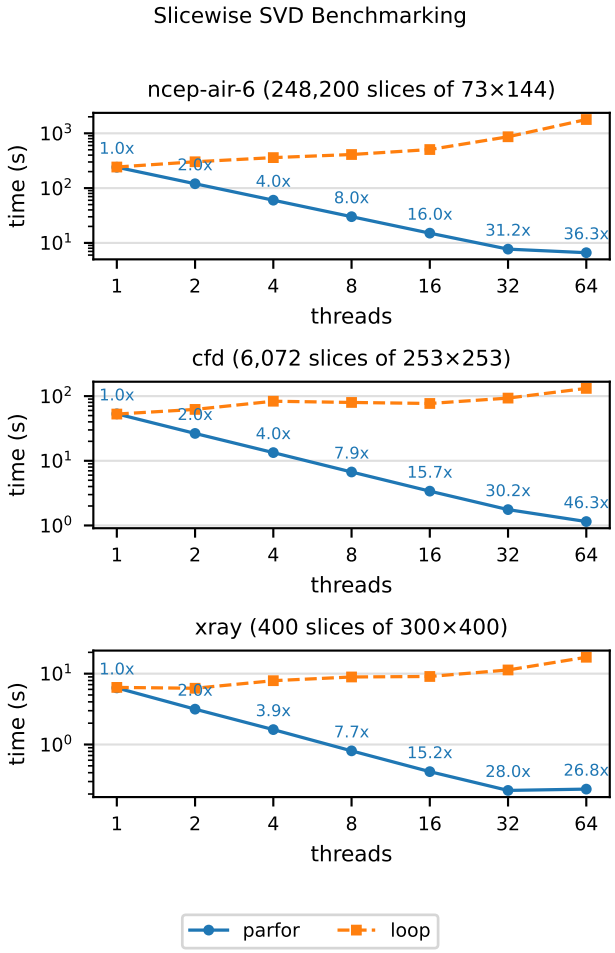


Figure 3: Slice-wise SVD wall time (parallel slices, sequential SVD vs. sequential slices, parallel SVD) for the `ncep-air-6`, `cfd`, and `xray` datasets.

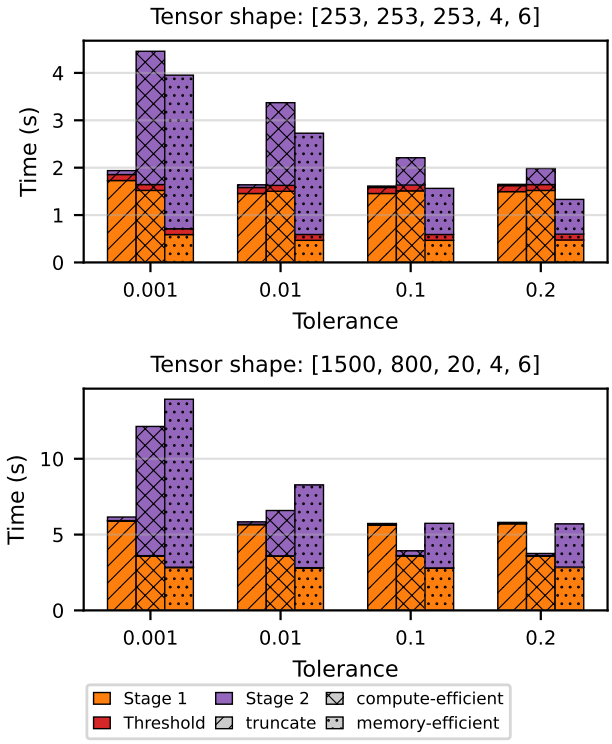


Figure 4: Breakdown times for the different t-SVDM-II strategies. Depending on the time and memory constraints, any of the three methods can be preferred.

t-SVDM (Algorithm 1) factorization, determine the threshold τ from all $n \cdot r$ singular values, and discard the singular vectors corresponding to values below τ. This is algorithmically simple and reuses the same slice-wise SVD kernel without modification, but pays the full cost of computing all singular vectors only to discard most of them when compression is aggressive. The `truncate` variant consumes the most intermediate memory of all three strategies storing up to 3× the memory footprint of the input data.

Choice 2: Compute-efficient. In the second strategy, a first pass computes only the singular values of each slice, without forming any singular vectors. The SVD algorithm consists of multiple steps (a possible QR factorization and bidiagonal reduction) before computing the singular values and vectors. We cache these intermediate entities. The global threshold τ and per-slice ranks ρ_i are determined from this pass. A second pass then computes only the leading ρ_i singular triplets for each slice, via a truncated SVD. We utilize the cached intermediaries to speed up the second SVD computation. All output buffers are formed to the exact required ranks before the second pass. The compressed factorization is stored in a variable-rank format in which the i-th slice's factor matrices occupy exactly $m \times \rho_i$ and $p \times \rho_i$ entries with no padding, as opposed to the dense $m \times r \times n$ and $p \times r \times n$ tensors produced by t-SVDM-I. This representation accurately reflects the storage cost of the compressed result and avoids allocating memory for singular components that are not retained. When target ranks are small, the savings in the second stage outweigh the cost of reading each slice twice. This `compute-efficient` version of t-SVDM-II reduces the memory footprint by half when compared to the `truncate` strategy. It also performs the fewest floating-point operations of all three strategies for most cases.

Choice 3: Memory-efficient. Finally, we consider the most memory efficient strategy, where we do not save any intermediaries from the preceding strategy. The second SVD computation could perform more computation than the `compute-efficient` strategy but we also save on the memory bandwidth needed in moving and copying the intermediaries. This strategy does not use any more space than that needed for holding the input and output buffers in memory. We call it the `memory-efficient` variant.

3.4.2 Performance of the three variants. Figure 4 shows the runtime breakdown of the different strategies on two data tensors for various relative-error tolerances. All times are averaged over 5 runs.

The first tensor is the `cfd` data ($253 \times 253 \times 253 \times 4 \times 6$, $\approx$ 3 GB, described in section 4.2.1) and the second is a larger synthetic tensor ($1500 \times 800 \times 20 \times 4 \times 6$, $\approx$ 4.3 GB) with a similar singular value distribution. No clear winner exists among the three strategies. The `truncate` strategy is most insensitive to the tolerance parameter as it computes the full t-SVDM every time. Its runtime is fairly consistent over all tolerances with the bulk of the time in the Stage 1 SVD. The `compute-efficient` and `memory-efficient` strategies show more differing behavior. The Stage 2 times for `compute-efficient` are always lower than those of `memory-efficient`, but the gain may not always be sufficient to overcome the time spent in moving the cached intermediaries in memory during the first stage. The Stage 1 SVD calls the `gesvd` kernel which employs the implicit zero-shift QR algorithm for its bidiagonal SVD (`bdsqr`), while the truncated SVD (`gesvdx`) calls the bisection plus inverse iteration algorithm (`bdsvdx`). Our studies showed an 8× performance difference between these methods, with `bdsqr` outperforming `bdsvdx`. This explains the relative competitiveness of the `truncate` strategy. Since we are concerned with data compression, we will use the `memory-efficient` strategy for all t-SVDM-II benchmarks.

4 Applications

In this section we demonstrate the practical utility of our work on three scientific datasets that are prohibitively large for currently available t-SVDM-I and t-SVDM-II implementations: atmospheric reanalysis data (Section 4.1), computational fluid dynamics data (Section 4.2), and X-ray diffuse scattering data (Section 4.3). While $\star_{\mathbf{M}}$ t-SVD have many applications, we focus on compression as it provides a direct and quantifiable measure of the quality of the decomposition. For each dataset we evaluate compression quality and strong scaling performance.

4.1 Climate Data (NCEP-Air)

4.1.1 Dataset. We use 10 years (1948–1957) of air temperature data from the NCEP/NCAR Reanalysis [21] at all 17 standard pressure levels, representing altitude. We represent the data as a 6-way tensor of shape $73 \times 144 \times 17 \times 4 \times 365 \times 10$ (`ncep-air-6`), where the first three modes are latitude, longitude, and pressure level, and the time axis is split into time-of-day (4), day-of-year (365), and year (10). February 29 observations are dropped from leap years to produce a uniform 365-day calendar, giving approximately 19.46 GB as `float64`.

4.1.2 Compression Quality. Figure 5 compares compression ratio across methods on the `ncep-air-6` tensor. t-SVDM-II-DCT achieves higher compression than t-SVDM-I-DCT at every error level, as per-slice variable truncation allows t-SVDM-II to discard many slices entirely. Both tensor methods outperform Empirical Orthogonal Functions(EOF) [15], the standard matrix-based compression in climate science. The result is consistent with the theoretical results of [26]. The DCT transform further outperforms the identity transform, as it concentrates energy into fewer singular values per slice. For the remaining experiments we focus on comparing t-SVDM-I and t-SVDM-II with the DCT transformation.

4.1.3 Strong Scaling. Figure 6 shows strong scaling of t-SVDM-I ($k = 1$, compression ratio $\approx$ 48.2×) and t-SVDM-II (tol=0.005, compression ratio $\approx$ 48.9×) on the `ncep-air-6` tensor. The t-SVDM-I algorithm achieves 42× speedup from 1 to 64 threads (147s → 3.5s) and t-SVDM-II algorithm achieves 27× (160s → 5.9s).

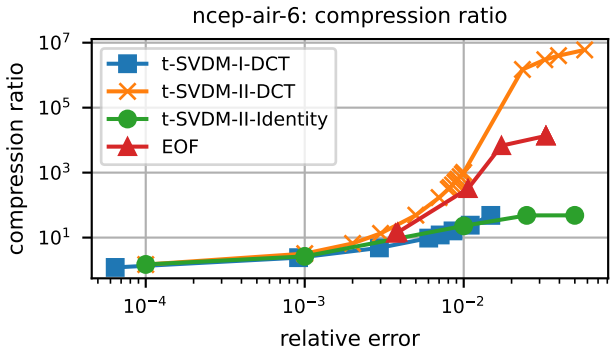


Figure 5: Compression ratio of the `ncep-air-6` tensor for different algorithms.

For t-SVDM-I, TTM and SVD computations scale similarly and nearly perfectly up to 32 threads. For the memory-efficient variant of t-SVDM-II, we expect the singular vectors computation to take longer than singular values. However the large number of empty slices present in this example makes the vectors computation cheaper. For t-SVDM-II, the *sequential* threshold calculation becomes the bottleneck as we scale, taking 25% of the time at 64 threads.

4.1.4 Extreme Event Preservation.

EOF vs. t-SVDM-II at extremes. A known limitation of EOF-based compression in climate science is that it can smooth extreme weather events [20]. Figure 7 compares the median pointwise relative error at extreme temperature events (top and bottom 0.5% of the time series at each grid point, at 850 hPa — a pressure level that approximates surface air temperature) for EOF and t-SVDM-II-DCT at a matched compression ratio of approximately 320×. For each spatial grid point, the error is computed over all time steps in the dataset and aggregated as the median across extreme events. Using DCT achieves lower reconstruction error at extremes across most of the globe, demonstrating that tensor-based compression better preserves physically significant rare events.

4.2 CFD

4.2.1 Dataset. We use a computational fluid dynamics (CFD) dataset generated from a direct numerical simulation of the Taylor–Green Vortex (TGV) [7], a standard benchmark for turbulent flow, solved using NekRS [14]. The simulation data is assembled into a 5-way tensor of shape $253 \times 253 \times 253 \times 4 \times 6$ (`cfd`, approximately 3 GB as `float64`), where the first three modes index the structured spatial grid, the fourth mode indexes four solution fields (velocity components u, v, w and pressure p), and the last mode indexes six flow snapshots. For visualization, we compute the z-component of vorticity, $\omega_z = \partial v/\partial x - \partial u/\partial y$, from the u and v velocity components of one snapshot; the result is shown in Figure 8.

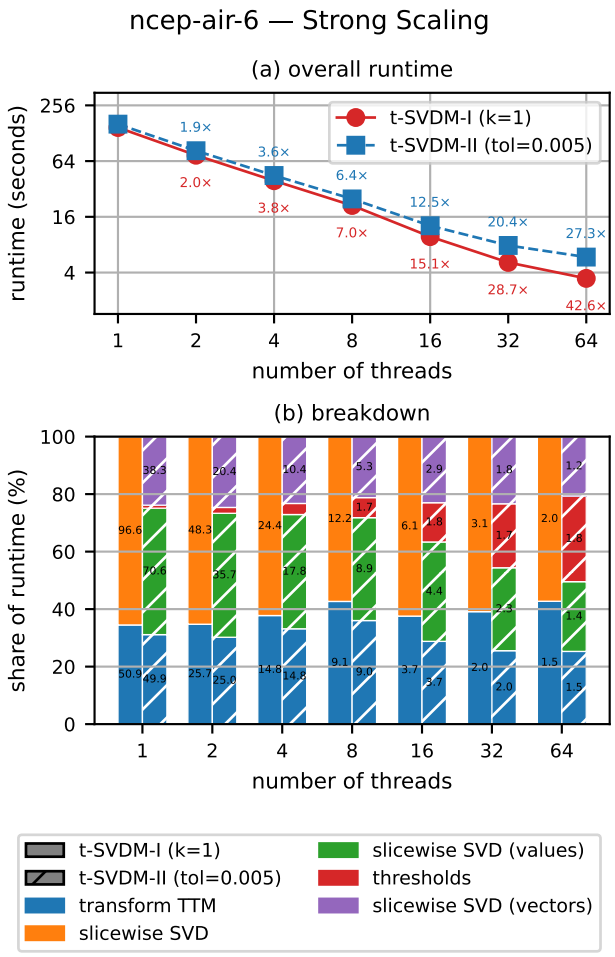


Figure 6: Strong scaling of both t-SVDM-I and t-SVDM-II for the `ncep-air-6` data with DCT transformation. t-SVDM-II is run with tolerance 0.005 (compression ratio ≈48.9), and t-SVDM-I is run with rank $k = 1$ (compression ratio ≈48.2). (a) shows total wall-clock runtime versus thread count, with speedup relative to 1 thread annotated at each point; (b) shows the same runtime broken down by computational stage, as a percentage of the total at each thread count, with most bar segments annotated with their raw runtime in seconds.

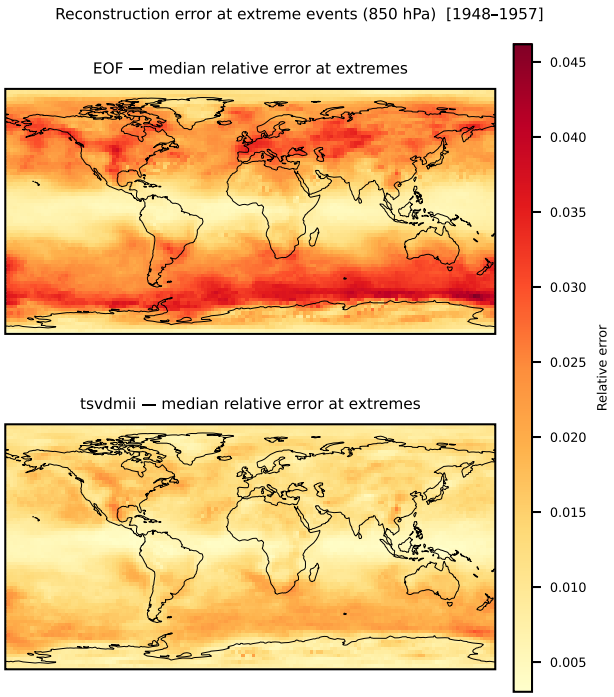


Figure 7: Median pointwise relative error at extreme temperature events (850 hPa) for EOF (top) and t-SVDM-II-DCT (bottom), at a matched compression ratio of ~320×. Lower is better.

4.2.2 Compression Quality. Figure 9 shows the compression ratio of t-SVDM-I and t-SVDM-II on the `cfd` tensor. At tight tolerances (relative error $\leq$ 0.05), the two algorithms achieve comparable compression ratios — for example, at relative error $\approx$ 0.05, t-SVDM-I ($k = 20$) achieves 6.3× while t-SVDM-II achieves 5.4×. However, as the tolerance is relaxed, t-SVDM-II pulls significantly ahead: at relative error $\approx$ 0.1, t-SVDM-II achieves 16× compression compared to 6.3× for t-SVDM-I, and the gap grows further at higher tolerances. Figure 8 shows the reconstructed vorticity field at tolerance 10^{-1}, confirming that the compressed reconstruction closely matches the original.

4.2.3 Strong Scaling. Figure 10 shows strong scaling of t-SVDM-I ($k = 20$, compression ratio $\approx$ 6.3×) and t-SVDM-II (tol=0.05, compression ratio $\approx$ 5.4×) on the `cfd` tensor. t-SVDM-I achieves a 39× speedup from 1 to 64 threads (43s → 1.1s), and t-SVDM-II achieves 34× (61s → 1.8s).

Similar to the climate data example, we see t-SVDM-I scale nearly perfectly to 32 threads with both TTM and SVD stages scaling similarly. For t-SVDM-II, we see the expected behavior of the memory-efficient variant with the singular vectors computation costing more than the singular values. This is in contrast to the climate example where we had a lot of empty slices. Notice that the singular vectors timing for t-SVDM-II is comparable to the SVD time for t-SVDM-I since they compute approximately the same number of singular vectors.

4.3 X-ray Crystallography

4.3.1 Dataset. We use a 3D X-ray diffuse scattering dataset measured on a $MoVO_2$ single crystal [28], stored as a 3-way tensor of shape $300 \times 400 \times 400$ (`xray`) in `float64` (approximately 366 MB), where each mode corresponds to one of the three orthogonal axes of the volumetric reciprocal-space measurement. A representative slice through the volume is shown in Figure 11.

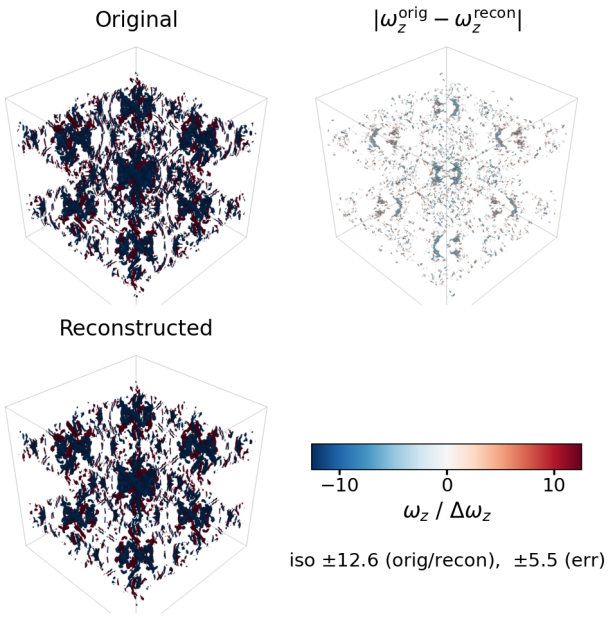


Figure 8: Iso-surfaces of the z-component of vorticity, $\omega_z = \partial v/\partial x - \partial u/\partial y$, for the Taylor–Green Vortex flow: original (top-left), compressed reconstruction (bottom-left), and pointwise error (top-right). Reconstruction is via t-SVDM-II-DCT at tolerance 10^{-1} (compression ratio $\approx 16\times$). Red and blue denote positive and negative values.

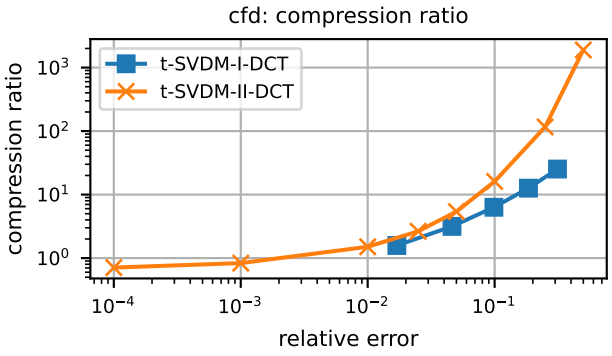


Figure 9: Compression ratio of the `cfd` tensor for different algorithms.

4.3.2 Compression Quality. Figure 12 shows the compression ratio of t-SVDM-I and t-SVDM-II on the `xray` tensor. t-SVDM-II achieves higher compression than t-SVDM-I at all error levels. Figure 11 shows the reconstructed slice at tolerance 10^{-2}, confirming that the reconstruction closely matches the original.

4.3.3 Strong Scaling. Figure 13 shows strong scaling of t-SVDM-I ($k = 150$, compression ratio $\approx 1.14\times$) and t-SVDM-II (tol=10^{-4}, compression ratio $\approx 1.19\times$) on the `xray` tensor. Both algorithms

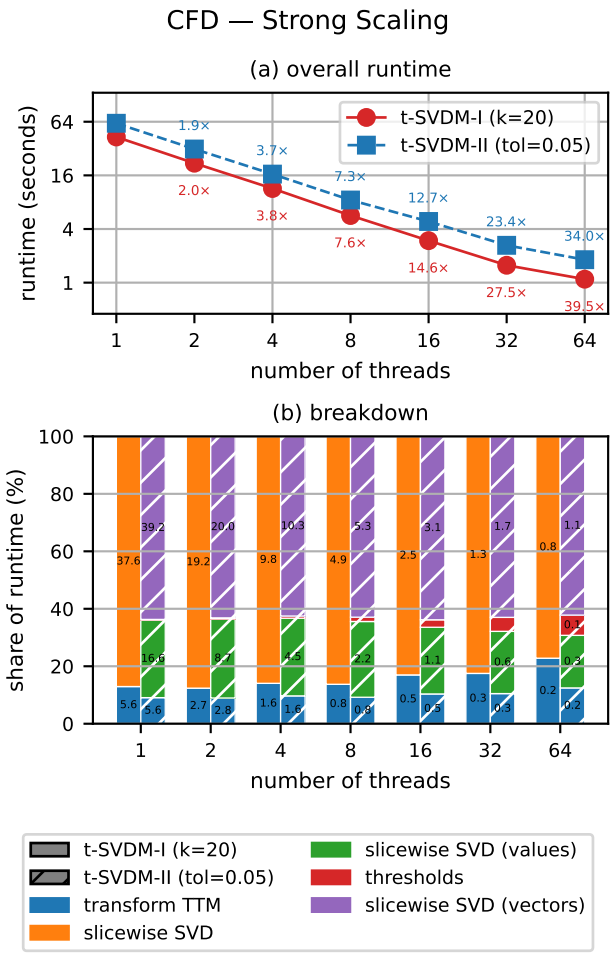


Figure 10: Strong scaling of both t-SVDM-I and t-SVDM-II for the `cfd` data with DCT transformation. t-SVDM-II is run with tolerance 0.05 (compression ratio ≈5.4), and t-SVDM-I is run with rank $k = 20$ (compression ratio ≈6.3). (a) shows total wall-clock runtime versus thread count, with speedup relative to 1 thread annotated at each point; (b) shows the same runtime broken down by computational stage, as a percentage of the total at each thread count, with most bar segments annotated with their raw runtime in seconds.

scale similarly — t-SVDM-I achieves 28× speedup from 1 to 64 threads (16.7s → 0.59s) and t-SVDM-II achieves 30× (18.7s → 0.63s). In both cases, slicewise SVD dominates the runtime.

For t-SVDM-I, SVD scales slightly better than TTM for this dataset. This could possibly be attributed to better memory layout for the SVD computation. The SVD slices are contiguous in memory whereas the third-mode TTM corresponds to a large row-wise matrix multiplication. The extra cost of t-SVDM-II is clearly due to the singular values and thresholding stages. This is also apparent for the `cfd` data and would likely be the case whenever the compression from t-SVDM-I and t-SVDM-II are comparable. Finally,

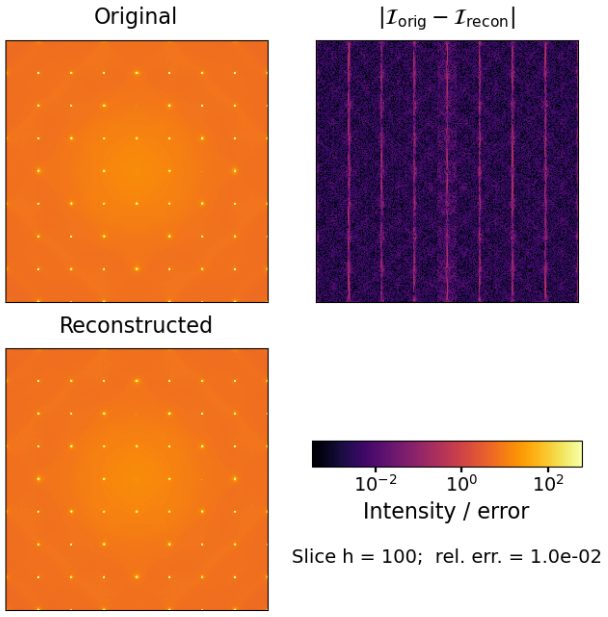


Figure 11: Two-dimensional slice through the X-ray diffuse-scattering volume: original (top-left), compressed reconstruction (bottom-left), and pointwise absolute error (top-right). Reconstruction is via t-SVDM-II-DCT at tolerance 10^{-2} (compression ratio $\approx$ 1.51×). A shared logarithmic colormap is used across all panels.

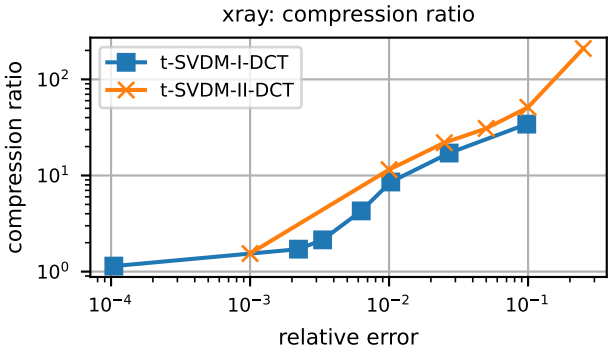


Figure 12: Compression ratio of the `xray` tensor for different algorithms.

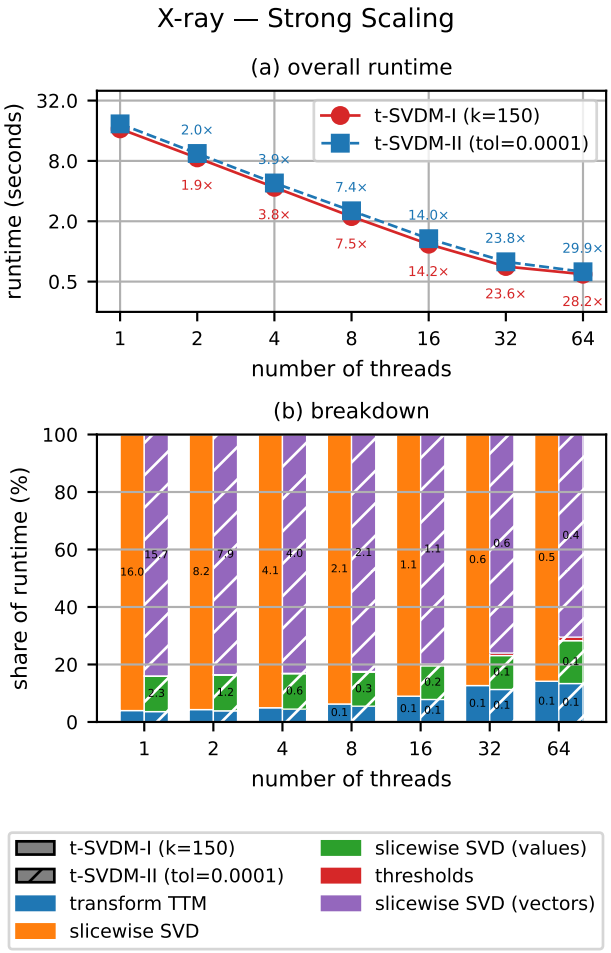


Figure 13: Strong scaling of both t-SVDM-I and t-SVDM-II for the `xray` data with DCT transformation. t-SVDM-II is run with tolerance 0.0001 (compression ratio ≈1.19), and t-SVDM-I is run with rank $k = 150$ (compression ratio ≈1.14). (a) shows total wall-clock runtime versus thread count, with speedup relative to 1 thread annotated at each point; (b) shows the same runtime broken down by computational stage, as a percentage of the total at each thread count, with most bar segments annotated with their raw runtime in seconds.

when compared to the `cfd` example the larger difference in cost between the singular vectors and values computations is due to the low compression achieved for the `xray` data.

5 Conclusions and Future Work

We present the first high-performance implementations of tensor decompositions in the $\star_{\mathbf{M}}$-framework. Our shared-memory parallel implementation takes advantage of the recent batch/strided interfaces developed for these linear algebra kernels. We achieve efficient scaling up to 64 threads on three real-world datasets from different scientific applications. Our case study on the climate dataset also highlights the utility of maintaining the multidimensionality of the dataset by preserving physically relevant rare events.

Our initial investigations open up many future directions for research. Supporting GPUs in the library is an immediate next step. The embarrassingly parallel nature of the slice-wise operations would greatly benefit from this development. The irregular data access pattern due to varying ranks in t-SVDM-II also poses some difficulty. Developing distributed-memory parallel algorithms will enable scaling to much larger dataset. Overcoming the difference

in data layout preferences for TTM and SVD stage without an explicit transpose operation is an immediate concern. Finally, we utilize explicit matrix multiplications in our TTM stage. Supporting efficient operator transforms is another avenue for future work.

Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, under Award Number DE-SC0025394 (M.T. Hussain, G. Ballard, A. Devarakonda) and under Contract DE-AC02-06CH11357 at Argonne National Laboratory (S. Eswar and V. Rao). N. Pesricha acknowledges a Master's Research Fellowship from the Ministry of Education, Government of India, and thanks Prof. Phani Motamarri (IISc Bengaluru) for valuable discussions and computational resources. We also thank Prof. Travis O'Brien (Indiana University) for discussions on climate data and Pinaki Pal (Argonne National Laboratory) for the CFD data.

References

[1] Ahmad Abdelfattah, Timothy Costa, Jack Dongarra, Mark Gates, Azzam Haidar, Sven Hammarling, Nicholas J. Higham, Jakub Kurzak, Piotr Luszczek, Stanimire Tomov, and Mawussi Zounon. 2021. A Set of Batched Basic Linear Algebra Subprograms and LAPACK Routines. *ACM Trans. Math. Softw.* 47, 3, Article 21 (June 2021), 23 pages. doi:10.1145/3431921

[2] Hussam Al Daas, Grey Ballard, and Peter Benner. 2022. Parallel Algorithms for Tensor Train Arithmetic. *SIAM Journal on Scientific Computing* 44, 1 (2022), C25–C53. doi:10.1137/20M1387158

[3] Woody Austin, Grey Ballard, and Tamara G. Kolda. 2016. Parallel Tensor Compression for Large-Scale Scientific Data. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 912–922. doi:10.1109/IPDPS.2016.67

[4] Grey Ballard, Alicia Klinvex, and Tamara G. Kolda. 2020. TuckerMPI: A Parallel C++/MPI Software Package for Large-Scale Data Compression via the Tucker Tensor Decomposition. *ACM Trans. Math. Software* 46, 2, Article 13 (June 2020), 31 pages. doi:10.1145/3378445

[5] Grey Ballard, Alicia Klinvex, and Tamara G. Kolda. 2020. TuckerMPI: A Parallel C++/MPI Software Package for Large-Scale Data Compression via the Tucker Tensor Decomposition. *ACM Trans. Math. Software* 46, 2 (2020), 1–31. doi:10.1145/3378445

[6] Grey Ballard and Tamara G. Kolda. 2025. *Tensor Decompositions for Data Science*. Cambridge University Press.

[7] Shivam Barwey, Pinaki Pal, Saumil Patel, Riccardo Balin, Bethany Lusch, Venkatram Vishwanath, Romit Maulik, and Ramesh Balakrishnan. 2025. Mesh-based super-resolution of fluid flows with multiscale graph neural networks. *Computer Methods in Applied Mechanics and Engineering* 443 (2025), 118072. doi:10.1016/j.cma.2025.118072

[8] J. Douglas Carroll and Jih-Jie Chang. 1970. Analysis of Individual Differences in Multidimensional Scaling via an N-way Generalization of "Eckart-Young" Decomposition. *Psychometrika* 35, 3 (1970), 283–319. doi:10.1007/BF02310791

[9] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. 2000. A Multilinear Singular Value Decomposition. *SIAM J. Matrix Anal. Appl.* 21, 4 (2000), 1253–1278. doi:10.1137/S0895479896305696

[10] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. 2000. On the Best Rank-1 and Rank-($R_1, R_2, \ldots, R_N$) Approximation of Higher-Order Tensors. *SIAM J. Matrix Anal. Appl.* 21, 4 (2000), 1324–1342. doi:10.1137/S0895479898346995

[11] Jack Dongarra, Mark Gates, Azzam Haidar, Jakub Kurzak, Piotr Luszczek, Stanimire Tomov, and Ichitaro Yamazaki. 2014. Accelerating Numerical Dense Linear Algebra Calculations with GPUs. *Numerical Computations with GPUs* (2014), 1–26. doi:10.1007/978-3-319-06548-9_1

[12] Carl Eckart and Gale Young. 1936. The Approximation of One Matrix by Another of Lower Rank. *Psychometrika* 1, 3 (1936), 211–218. doi:10.1007/BF02288367

[13] Srinivas Eswar, Koby Hayashi, Grey Ballard, Ramakrishnan Kannan, Michael A. Matheson, and Haesun Park. 2021. PLANC: Parallel Low-Rank Approximation with Nonnegativity Constraints. *ACM Trans. Math. Software* 47, 3, Article 20 (June 2021), 37 pages. doi:10.1145/3432185

[14] Paul Fischer, Stefan Kerkemeier, Misun Min, Yu-Hsiang Lan, Malachi Phillips, Thilina Rathnayake, Elia Merzari, Ananias Tomboulides, Ali Karakus, Noel Chalmers, and Tim Warburton. 2022. NekRS, a GPU-accelerated spectral element Navier–Stokes solver. *Parallel Comput.* 114 (2022), 102982. doi:10.1016/j.parco.2022.102982

[15] A. Hannachi, I. T. Jolliffe, and D. B. Stephenson. 2007. Empirical Orthogonal Functions and Related Techniques in Atmospheric Science: A Review. *International Journal of Climatology* 27, 9 (2007), 1119–1152. doi:10.1002/joc.1499

[16] Richard A. Harshman. 1970. Foundations of the PARAFAC Procedure: Models and Conditions for an "Explanatory" Multi-modal Factor Analysis. *UCLA Working Papers in Phonetics* 16 (1970), 1–84.

[17] Ahmed E. Helal, Jan Laukemann, Fabio Checconi, Jesmin Jahan Tithi, Teresa Ranadive, Fabrizio Petrini, and Jeewhan Choi. 2021. ALTO: adaptive linearized storage of sparse tensors. In *Proceedings of the 35th ACM International Conference on Supercomputing (ICS '21)*. Association for Computing Machinery, New York, NY, USA, 404?416. doi:10.1145/3447818.3461703

[18] Frank L. Hitchcock. 1927. The Expression of a Tensor or a Polyadic as a Sum of Products. *Journal of Mathematics and Physics* 6, 1–4 (1927), 164–189. doi:10.1002/sapm192761164

[19] Intel Corporation. 2026. *Developer Reference for Intel® oneAPI Math Kernel Library*. https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-documentation.html

[20] Yujing Jiang, Daniel Cooley, and Michael F. Wehner. 2020. Principal Component Analysis for Extremes and Application to U.S. Precipitation. *Journal of Climate* 33, 15 (2020). doi:10.1175/JCLI-D-19-0413.1

[21] E. Kalnay, M. Kanamitsu, R. Kistler, W. Collins, D. Deaven, L. Gandin, M. Iredell, S. Saha, G. White, J. Woollen, Y. Zhu, M. Chelliah, W. Ebisuzaki, W. Higgins, J. Janowiak, K. C. Mo, C. Ropelewski, J. Wang, A. Leetmaa, R. Reynolds, R. Jenne, and D. Joseph. 1996. The NCEP/NCAR 40-year reanalysis project. *Bulletin of the American Meteorological Society* 77, 3 (1996), 437–472. doi:10.1175/1520-0477(1996)077<0437:TNYRP>2.0.CO;2

[22] O. Kaya and B. Uçar. 2016. High Performance Parallel Algorithms for the Tucker Decomposition of Sparse Tensors. In *45th International Conference on Parallel Processing (ICPP '16)*. 103–112. doi:10.1109/ICPP.2016.19

[23] Katherine Keegan and Elizabeth Newman. 2025. Projected Tensor-Tensor Products for Efficient Computation of Optimal Multiway Data Representations. *Linear Algebra Appl.* 729 (2025), 100–147. doi:10.1016/j.laa.2025.09.018

[24] Katherine Keegan, Tanvi Vishwanath, and Yihua Xu. 2021. A Tensor SVD-based Classification Algorithm Applied to fMRI Data. arXiv:2111.00587 [cs.LG] https://arxiv.org/abs/2111.00587

[25] Eric Kernfeld, Misha Kilmer, and Shuchin Aeron. 2015. Tensor-Tensor Products with Invertible Linear Transforms. *Linear Algebra Appl.* 485 (2015), 545–570. doi:10.1016/j.laa.2015.07.021

[26] Misha E Kilmer, Lior Horesh, Haim Avron, and Elizabeth Newman. 2021. Tensor-tensor algebra for optimal representation and compression of multiway data. *Proceedings of the National Academy of Sciences* 118, 28 (2021), e2015851118. doi:10.1073/pnas.2015851118

[27] Tamara G. Kolda and Brett W. Bader. 2009. Tensor Decompositions and Applications. *SIAM Rev.* 51, 3 (2009), 455–500. doi:10.1137/07070111X

[28] Wei Kuang, Vishwas Rao, Alexis Montoison, François Pacaud, and Mihai Anitescu. 2025. Recovering Sparse DFT from Missing Signals via Interior Point Method on GPU. *arXiv preprint arXiv:2502.04217* (2025).

[29] Canyi Lu. 2018. *Tensor-Tensor Product Toolbox*. Carnegie Mellon University. https://github.com/canyilu/tproduct.

[30] Leon Mirsky. 1960. Symmetric Gauge Functions and Unitarily Invariant Norms. *The Quarterly Journal of Mathematics* 11, 1 (1960), 50–59. doi:10.1093/qmath/11.1.50

[31] Elizabeth Newman and Katherine Keegan. 2025. Optimal Matrix-Mimetic Tensor Algebras via Variable Projection. *SIAM J. Matrix Anal. Appl.* 46, 3 (2025), 1764–1790. arXiv:https://doi.org/10.1137/24M1702635 doi:10.1137/24M1702635

[32] Ivan V. Oseledets. 2011. Tensor-Train Decomposition. *SIAM Journal on Scientific Computing* 33, 5 (2011), 2295–2317. doi:10.1137/090752286

[33] Eric T. Phipps and Tamara G. Kolda. 2019. Software for Sparse Tensor Decomposition on Emerging Computing Architectures. *SIAM Journal on Scientific Computing* 41, 3 (2019), C269–C290. doi:10.1137/18M1210691

[34] Melven Röhrig-Zöllner, Jonas Thies, and Achim Basermann. 2022. Performance of the Low-Rank TT-SVD for Large Dense Tensors on Modern MultiCore CPUs. *SIAM Journal on Scientific Computing* 44, 4 (2022), C287–C309. doi:10.1137/21M1395545

[35] Shaden Smith and George Karypis. 2016. A Medium-Grained Algorithm for Distributed Sparse Tensor Factorization. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 902–911. doi:10.1109/IPDPS.2016.113 SPLATT: distributed-memory parallel CP decomposition.

[36] Ledyard R. Tucker. 1966. Some Mathematical Notes on Three-mode Factor Analysis. *Psychometrika* 31, 3 (1966), 279–311. doi:10.1007/BF02289464

[37] Sasindu Wijeratne, Rajgopal Kannan, and Viktor Prasanna. 2025. AMPED: Accelerating MTTKRP for Billion-Scale Sparse Tensor Decomposition on Multiple GPUs. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP '25)*. Association for Computing Machinery, New York, NY, USA, 208–217. doi:10.1145/3754598.3754651